Weichi Zhao

AI Infrastructure / LLM Serving Platform Engineer

Hangzhou, China   wzhao4z@outlook.com   GitHub: WZStephen   LinkedIn   Homepage: wzstephen.github.io

Target roles: LLM serving platform, OpenAI-compatible MaaS, AI gateway, agent platform, heterogeneous GPU/XPU scheduling.

Profile

AI infrastructure engineer focused on production LLM serving platforms and model-service control planes. I build OpenAI-compatible MaaS capabilities, public model services, offline batch inference, model gateways, agent-client integration, routing/orchestration reliability, and Kubernetes-native serving on heterogeneous GPU/XPU clusters. In current MaaS and LLM serving work, I embed Claude Code, Codex, OpenCode, OpenClaw, and similar agent tools into code search, debugging, documentation, release checks, and verification loops to iterate faster on complex platform changes.

Impact Highlights

- Scaled BatchFlow for production data distillation and batch generation workloads, later supporting around 10M requests per day with success rate above 99%.
- Validated BatchFlow -> SGLang Router -> multi-KLX backend for Qwen3: about 300K requests / 9.75B tokens, about 99.9% success rate, and below 5% BatchFlow/Router CPU and memory overhead.
- Built OpenAI-compatible MaaS public-service capabilities around API keys, model access, TPM/RPM limits, token budgets, model mapping, and reusable onboarding flows for agent clients.
- Applied agent-assisted workflows in current MaaS/LLM serving work for code search, incident writeups, design drafts, release checks, and knowledge-base updates, turning repeated manual steps into reusable iteration loops.
- Hardened serving reliability across gateway, Router, Worker, Service discovery, readiness, and long-connection failure modes instead of relying only on Pod-level health.

Core Skills

LLM Serving	SGLang, vLLM, OpenAI-compatible APIs, streaming, embeddings, Responses API, PD disaggregation, KV/prefix cache, cache-aware and bucket routing.
MaaS Public Service	Higress, LiteLLM, AI Gateway, API keys, team/model access, TPM/RPM limits, token budgets, model mapping, observability.
Control Plane	Kubernetes operators/controllers, CRDs, Helm, Kustomize, readiness, finalizers, service discovery, rollout safety, retry and rollback workflows.
Agent Platform	Claude Code / Codex / OpenCode integration, tool-calling API compatibility, OpenAI/Anthropic-style adapters, long-context cache optimization, agent-assisted engineering iteration.
Engineering	Go, Python, Linux, Prometheus metrics, MySQL/SQLite/Redis, distributed systems debugging, performance and capacity testing.

Experience

Zhejiang Lab, High-Performance Computing Facility Research Center — 2024 - Present

Senior Research Specialist, AI infrastructure and MaaS platform

- Built and operated LLM serving platform capabilities across CPU ACK and GPU ACK clusters, covering public model-service entrypoints, OpenAI-compatible APIs, Higress/LiteLLM gateways, SGLang Router based model routing, IGD/IGDSA inference orchestration, BatchFlow offline inference, and Agent-facing API compatibility.
- Owned production troubleshooting and hardening for LLM serving paths, including Router worker registration convergence, Service/Endpoint readiness, PD prefill/decode topology, CrashLoop and long-connection failures, gateway header/model mapping issues, and multi-cluster rollout validation.
- Designed MaaS-Controller and optimizer-loop mechanisms around LiteLLM, Router, IGDSA, BatchFlow, and gateway metrics, with public-model limit adjustment, offline concurrency tuning, observation windows, rollback criteria, and online-SLA-first rules.

Zhejiang Lab, Intelligent Supercomputing Research Center — Aug 2021 - 2023

Engineering Specialist, cloud-native AI and HPC platforms

- Worked on cloud-native AI/HPC platform projects, including Kubernetes adaptation on domestic OS and accelerators, device plugin customization for accelerator availability, PyTorch/model adaptation, and distributed training framework exploration.
- Supported model training and inference platform adaptation in domestic heterogeneous hardware/software environments, covering job submission, resource discovery, container runtime validation, and baseline performance verification.

Selected AI Infrastructure Projects

BatchFlow - OpenAI-compatible offline inference platform — **LLM batch inference**

Asynchronous JSONL batch execution for large-scale, non-real-time model workloads.

- Designed Batch API compatible capabilities: file upload, batch lifecycle, cancellation, result/error files, streaming file processing, request-level error isolation, and backend concurrency control.
- Refactored BatchFlow toward multi-replica operation with Kubernetes leader election, sharded workers, header forwarding, upload limits, stale-file cleanup, Kustomize upgrades, monitoring, and failure-tolerant finalization.

- Supported production data-distillation and batch-generation workloads, later reaching daily processing around 10M requests with success rate above 99% under controlled concurrency.
- Validated a BatchFlow -> SGLang Router -> multi-KLX path for Qwen3-235B-A22B-Thinking-2507 at about 300K requests, about 9.75B tokens, about 99.9% success rate, and below 5% BatchFlow/Router CPU/memory overhead.

SGLang Router, IGD, and production serving reliability — **Routing and orchestration**

Reliable worker registration, routing, and recovery for multi-worker and PD-disaggregated inference.

- Improved Router worker synchronization by reconciling desired workers against live `/workers` state and requeueing on missing, extra, or failed registrations.
- Enhanced IGD/ISI startup and readiness with Pod-IP early registration, `publishNotReadyAddresses`, preserved Service finalizers, Router metrics Service generation, and ISI-level auto-restart with retry budgets.
- Led production RCA for ARG_MAX overflow from service-link environment variables, Router not managed after Worker Ready, stale Service selectors, and long-connection timeout behavior.

MaaS public model service and agent integration optimization — **OpenAI-compatible API / Agent runtime**

Unified model access, authentication, rate limiting, protocol compatibility, cost governance, and troubleshooting for internal users, agent clients, and offline workloads.

- Built OpenAI-compatible public model-service entrypoints covering Chat Completions, Responses-style APIs, embeddings, Batch API, and agent-client access while hiding differences across model backends, routers, and heterogeneous clusters.
- Contributed public-service governance around API keys, teams, model access, quotas, TPM/RPM limits, token budgets, model mapping, and cost-control policies for shared model access.
- Analyzed protocol compatibility boundaries for Claude Code / Codex-style agent clients, including system prompts, tool schemas, tool results, streaming events, Responses-style APIs, and OpenAI/Anthropic field differences.
- Diagnosed low SGLang prefix/KV cache hit rates for Claude Code long-context requests by comparing rendered prompts, cached-token logs, and token longest common prefix; identified dynamic attribution fields that polluted request prefixes and mitigated the issue.
- Embedded agent tools into current engineering workflows for cross-repository code navigation, log summarization, design drafts, PR/issue writing, release checks, and knowledge-base updates, shortening feedback loops for complex platform tasks.
- Established an agent-to-MaaS troubleshooting path across API keys, model permissions, quotas, max_tokens, rate limits, connection timeouts, gateway forwarding, Router/Worker state, and model protocol behavior.

Kubernetes device plugin and heterogeneous accelerator platform — **GPU/XPU scheduling**

Cloud-native resource discovery, scheduling, and validation for domestic heterogeneous accelerator environments.

- Customized device-plugin and resource-management paths for accelerator availability, node resource reporting, container runtime validation, card-level scheduling, and failure isolation.
- Built deployment, upgrade, rollback, and E2E validation procedures for Kubernetes AI/HPC environments on domestic operating systems and accelerator stacks.

Earlier Projects

Arizona State University, SNAC Lab — Jan 2020 - Aug 2020
Virtual platform developer and research assistant

- Developed modules for THOTH Lab, an open virtual lab platform for education and research, including SDN/OpenFlow lab modules and AWS instance control through Boto3-backed REST APIs.
- Implemented an Android prototype for an IR-CP-ABE based network security research system.

Arizona State University, CIDSE — Aug 2019 - Dec 2020
Graduate service assistant

- Assisted CSE205 Object-Oriented Programming, CSE412 Database Management, CSE434 Computer Networks, and CSE464 Software QA and Testing.

Education

Arizona State University — May 2019 - Dec 2020
Master of Computer Science, GPA 3.97/4.00

Arizona State University — Aug 2015 - May 2019
Bachelor of Science in Computer Science, GPA 3.40/4.00

Last updated: June 2026